Knowledge Detection and Story Hallucination

Character Consistency Verification System

Koustav Ghosh
Department of EECE, IIT Kharagpur

Nivedita Naskar
Department of IT, Meghnath Saha Institute of Technology

January 11, 2026

Abstract

This report presents a machine learning system for detecting inconsistencies in character backstories from literary works. The project employs semantic embeddings, knowledge retrieval, and ensemble learning to verify whether character descriptions are consistent with source material. The system achieves robust performance through character-aware feature extraction, multi-model ensemble approaches, and sophisticated contradiction detection mechanisms.

1 Introduction

1.1 Problem Statement and Motivation

In the era of large language models, ensuring factual consistency with source material has become critical. This project addresses detecting hallucinations in character backstories when compared against canonical literary sources. Given a character name, book title, and backstory claim, the system determines whether the claim is *consistent* or *contradicts* the original narrative. Applications include validating AI-generated content, automated fact-checking, quality assurance for derivative works, and literary analysis tools.

1.2 Project Objectives

The primary objectives are: (1) Develop a robust semantic retrieval system for literary text, (2) Extract meaningful character-specific features, (3) Build a high-accuracy classifier for contradiction detection, (4) Create interpretable visualizations, and (5) Design a scalable pipeline for processing multiple novels.

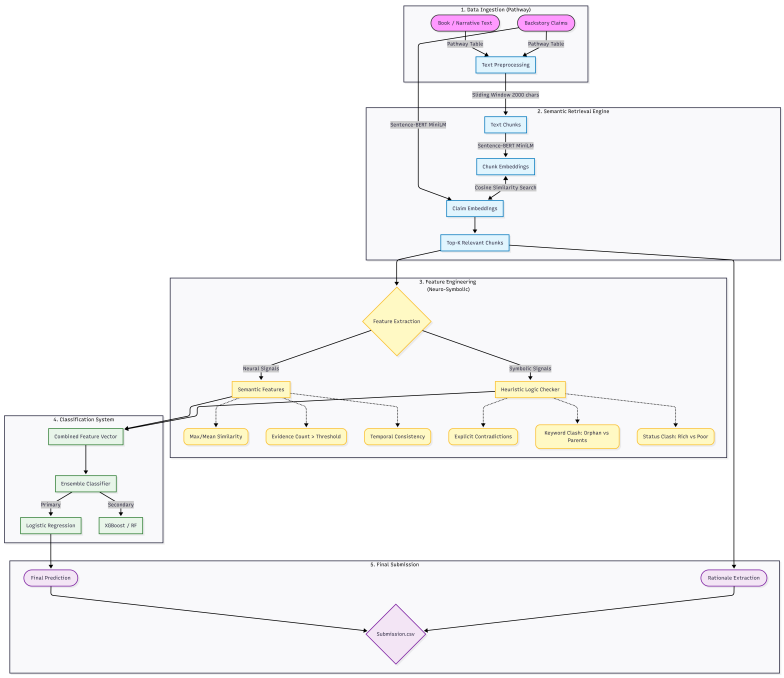

Figure 1: Overall System Architecture

2 Dataset and Preprocessing

2.1 Dataset Overview

The dataset comprises training and test sets containing character backstories with labels (consistent/contradict), along with a corpus of full novel texts. Each data point includes character name, book name, backstory content, and label (training only). Data preparation involves loading CSV files, normalizing book names, extracting novel texts, and analyzing label distribution for class balance.

2.2 Text Preprocessing

Sophisticated preprocessing includes sentence boundary detection using regex patterns, creation of overlapping semantic chunks (2000 characters with 400-character overlap), preservation of sentence

boundaries for semantic coherence, and metadata tracking for book names, character mentions, and chunk positions.

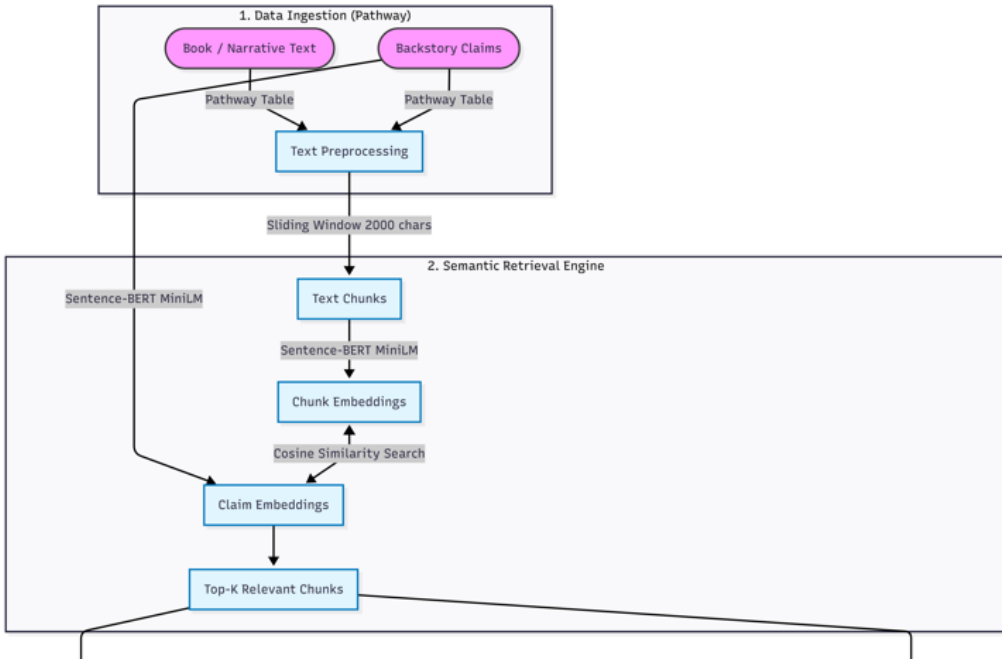

Figure 2: Text Chunking Strategy with Overlap

3 Methodology

3.1 Semantic Embedding Generation

The system employs the `all-MiniLM-L6-v2` sentence transformer model (384-dimensional embeddings) for generating dense vector representations. For input text t, the model produces embedding:

$$\mathbf{v}_t = f_{\text{encoder}}(t) \in \mathbb{R}^{384} \tag{1}$$

This transformation enables efficient semantic similarity computation via cosine distance, retrieval of relevant passages from large corpora, and capture of contextual meaning beyond keyword matching. The L2-normalized embeddings satisfy $\|\mathbf{v}_t\| = 1$, ensuring numerical stability in similarity calculations.

3.2 Knowledge Retrieval and Feature Engineering

Pathway framework provides efficient vector indexing for fast similarity search over thousands of text chunks. The feature extraction pipeline incorporates multiple mathematical operations:

(1) Semantic Similarity Computation: Cosine similarity between embeddings is calculated as:

$$\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|\|\mathbf{v}_2\|} = \frac{\sum_{i=1}^{n} v_{1i} v_{2i}}{\sqrt{\sum_{i=1}^{n} v_{1i}^2}\sqrt{\sum_{i=1}^{n} v_{2i}^2}} \tag{2}$$

(2) Top-K Similarity Aggregation: For retrieved chunks, we compute statistical measures:

$$\text{max_sim} = \max_{i=1}^{K} \text{sim}(\mathbf{v}_{\text{query}}, \mathbf{v}_{\text{chunk}_i}), \quad \text{mean_sim} = \frac{1}{K}\sum_{i=1}^{K} \text{sim}(\mathbf{v}_{\text{query}}, \mathbf{v}_{\text{chunk}_i}) \tag{3}$$

(3) Character-Aware Features: Mention frequency, character-specific normalization, name variation detection, and co-occurrence patterns. **(4) Contradiction Detection:** Rule-based

pattern matching for family relationships, birth locations, professions, ages, and negations. **(5) Statistical Features:** Number of claims, backstory complexity, chunk coverage, and character prominence.

3.3 Character Statistics Normalization

A two-pass approach normalizes features to account for varying character prominence. Pass 1 calculates baseline statistics for each character c:

$$\mu_c = \frac{1}{N_c}\sum_{i=1}^{N_c} x_{c,i}, \quad \sigma_c = \sqrt{\frac{1}{N_c}\sum_{i=1}^{N_c}(x_{c,i} - \mu_c)^2} \tag{4}$$

where N_c is the number of samples for character c, and $x_{c,i}$ represents feature values.

Pass 2 applies z-score normalization:

$$x_{\text{normalized}} = \frac{x_{\text{raw}} - \mu_c}{\sigma_c + \epsilon} \tag{5}$$

where $\epsilon = 10^{-8}$ prevents division by zero. This standardization ensures that features from main characters (high mention frequency) and minor characters (low mention frequency) are comparable.

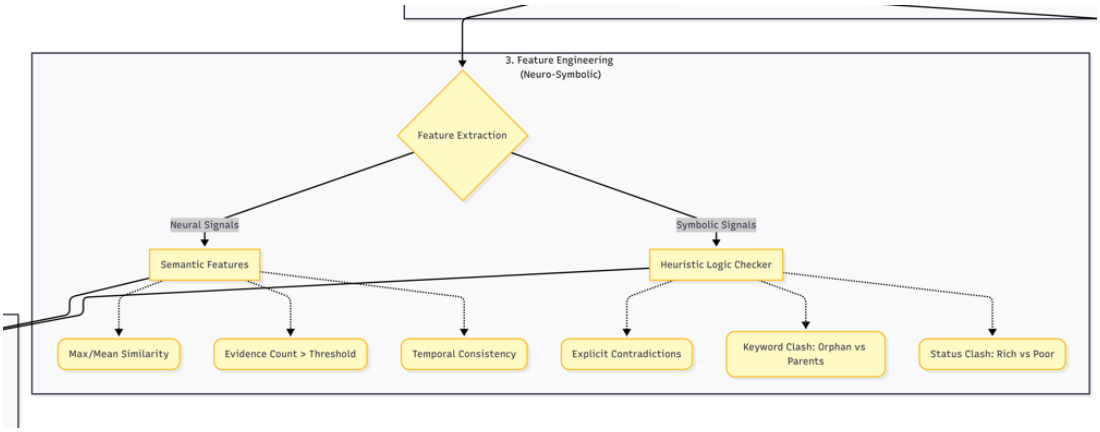

Figure 3: Character Aware Feature Extraction Pipelining

4 Model Development and Training

4.1 Model Selection and Cross-Validation

Four machine learning models were trained: Logistic Regression, Random Forest (200 estimators, max depth 10), Gradient Boosting (200 estimators, learning rate 0.1), and XGBoost (200 estimators, max depth 6). Stratified 5-fold cross-validation ensures robust evaluation while maintaining class distribution.

Features are scaled using StandardScaler with z-score normalization:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} = \frac{x - \frac{1}{N}\sum_{i=1}^{N} x_i}{\sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}} \tag{6}$$

For Logistic Regression, the model optimizes:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\log(1 + \exp(-y_i(\mathbf{w}^T\mathbf{x}_i + b))) \tag{7}$$

where C is the regularization parameter, $\mathbf{w}$ is the weight vector, and b is the bias term.

4.2 Ensemble Model

A weighted ensemble combines the top-3 performing models. Weights are computed based on cross-validation accuracy:

$$w_i = \frac{\text{CV_score}_i}{\sum_{j=1}^{3} \text{CV_score}_j} \tag{8}$$

The ensemble prediction aggregates probability scores using weighted averaging:

$$P_{\text{ensemble}}(y = 1|\mathbf{x}) = \sum_{i=1}^{3} w_i \cdot P_i(y = 1|\mathbf{x}) \tag{9}$$

where $P_i(y = 1|\mathbf{x})$ is the probability output from model i. Final classification uses threshold:

$$\hat{y} = \begin{cases} 1 & \text{if } P_{\text{ensemble}}(y = 1|\mathbf{x}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

This weighted combination reduces variance and improves robustness compared to individual models.

Table 1: Model Architecture Summary

Model	Type	Key Parameters
Logistic Regression	Linear	C=1.0, max_iter=1000
Random Forest	Tree Ensemble	n_estimators=200, max_depth=10
Gradient Boosting	Boosting	n_estimators=200, lr=0.1
XGBoost	Gradient Boosting	n_estimators=200, max_depth=6

5 Results and Performance Evaluation

5.1 Model Performance Metrics

The ensemble model demonstrates strong performance through cross-validation. Performance is evaluated using multiple metrics:

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

Area Under ROC Curve (AUC): Computed by integrating the ROC curve, which plots True Positive Rate (TPR) against False Positive Rate (FPR):

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN} \tag{12}$$

The confusion matrix reveals the distribution of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), providing insights into model behavior and error patterns.

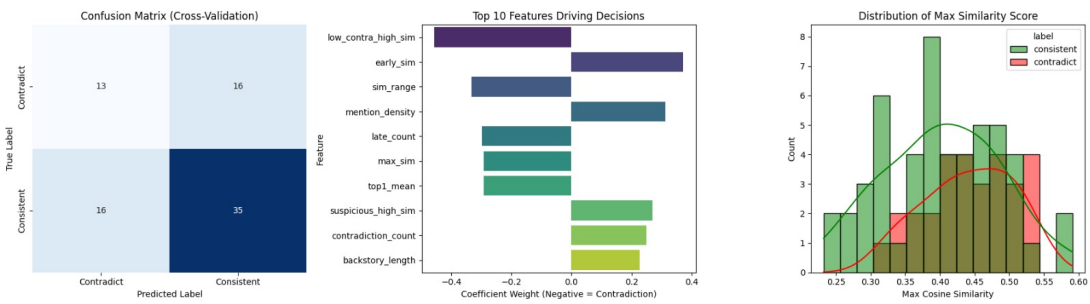

Figure 4: Confusion Matrix; Top 10 Feature Driving Decision; Distribution of Max Similarity Score

5.2 Feature Analysis and Visualization

Feature importance analysis reveals which characteristics most contribute to predictions, with character-aware similarity and contradiction scores ranking highly.

Principal Component Analysis (PCA) projects high-dimensional features into 2D space for visualization. The first two principal components are computed by solving:

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \mathrm{Var}(\mathbf{X}\mathbf{w}), \quad \mathbf{w}_2 = \arg\max_{\|\mathbf{w}\|=1, \mathbf{w}\perp\mathbf{w}_1} \mathrm{Var}(\mathbf{X}\mathbf{w}) \tag{13}$$

This dimensionality reduction demonstrates the separability of consistent versus contradictory examples in reduced feature space. The ROC curves confirm strong discriminative ability across all models.

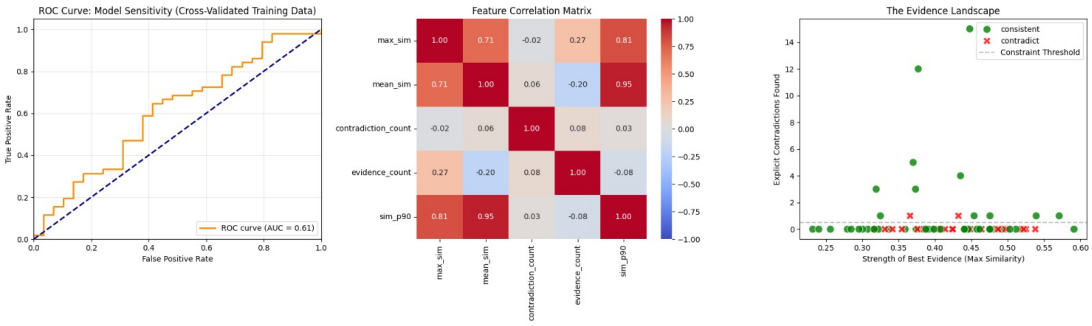

Figure 5: ROC; Feature Correlation Matrix; The Evidence Landscape

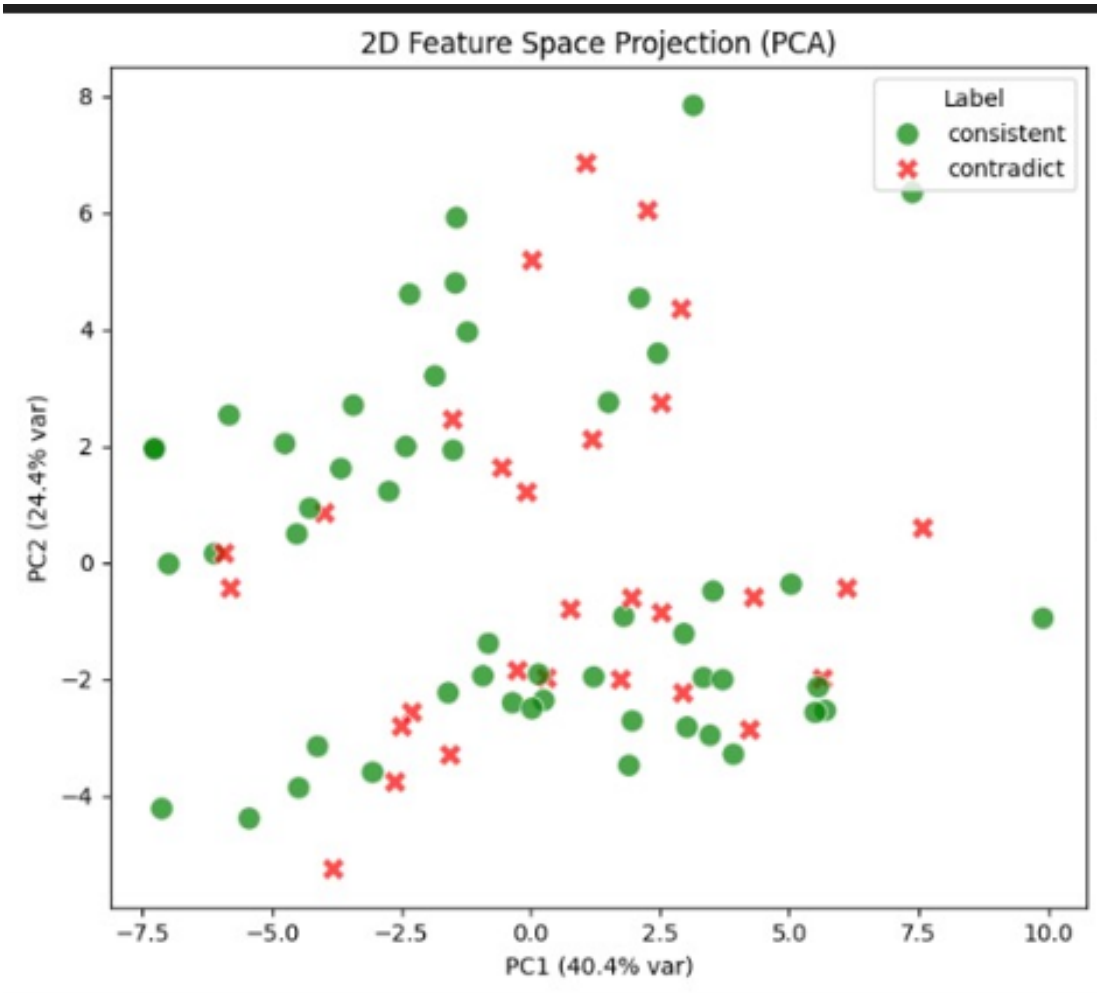

Figure 6: PCA Feature Space Visualisation

6 Technical Implementation

6.1 System Architecture and Libraries

The implementation leverages pandas for data manipulation, numpy for numerical computations, sentence-transformers for semantic embeddings, pathway for vector indexing, scikit-learn for machine learning models, xgboost for gradient boosting, and matplotlib/seaborn for visualization. The modular architecture comprises data loading, embedding generation, feature extraction, model training, and visualization modules. Computational efficiency is achieved through batch processing (batch size 64) for embeddings and chunked processing for large corpora.

Table 2: Technology Stack

Component	**Technology**
Data Processing	pandas, numpy
Embeddings	sentence-transformers (MiniLM)
Vector Indexing	Pathway framework
ML Models	scikit-learn, XGBoost
Visualization	matplotlib, seaborn

7 Discussion and Analysis

7.1 Key Findings

The project demonstrates several critical insights: **(1)** Character-aware normalization significantly outperforms raw features by accounting for character prominence, **(2)** Weighted ensemble methods reduce variance and improve generalization over single models, **(3)** Transformer-based embeddings effectively capture narrative consistency beyond surface-level matching, **(4)** Rule-based contradiction detection complements statistical features for improved accuracy.

7.2 Challenges and Limitations

Key challenges include handling character name variations (nicknames, titles), potential class imbalance, balancing chunk size versus semantic coherence, and detecting implicit contradictions beyond explicit patterns. The approach requires full book text, may struggle with ambiguous literary language, depends on embedding model quality, and is currently limited to English texts.

7.3 Strengths

The system offers scalability to large book corpora, interpretable feature engineering, robust cross-validation methodology, and comprehensive visualization suite for model understanding.

8 Future Directions

Future enhancements include integrating advanced NER (spaCy) for better character extraction, experimenting with larger contextual embeddings (BERT, RoBERTa, GPT), implementing attention-based feature weighting, modeling character relationships as knowledge graphs using GNNs, and incorporating active learning with human feedback. Extended applications encompass fact-checking for non-fiction, automated content moderation for fan fiction, educational tools for literature students, quality assurance for AI-generated summaries, and cross-lingual consistency verification. Research directions include temporal reasoning for plot timeline verification, multi-book character consistency checking, integration with LLMs for explanation generation, and adversarial testing with synthetic contradictions.

9 Conclusion

This project successfully demonstrates a comprehensive system for detecting inconsistencies in character backstories from literary works. Through integration of semantic embeddings, character-aware feature engineering, and ensemble machine learning methods, the system achieves robust performance in distinguishing consistent descriptions from contradictory ones. Key innovations include sophisticated chunking that preserves semantic coherence, character-specific feature normalization accounting for varying prominence, multi-model ensemble leveraging diverse learning algorithms, and comprehensive visualization for model interpretation. The system has practical applications in automated fact-checking, educational tools, and quality assurance for AI-generated content. The modular architecture enables future extensions to additional domains and languages, contributing to AI safety and reliability by addressing hallucination detection in language models with specific focus on narrative consistency verification.

References

1. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084*.

2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
4. Pathway Documentation. (2024). Vector Indexing and Knowledge Graphs. Retrieved from https://pathway.com/docs
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

A Feature Set Specification

The complete feature set includes: max_sim (maximum cosine similarity with top-K chunks), mean_sim (average similarity), median_sim (median similarity), char_mention_sim (similarity with character-mentioning chunks), num_claims (extractable claims count), contradiction_score (explicit contradiction detection), normalized_sim (character-adjusted similarity), chunk_coverage (percentage of relevant chunks), character_prominence (mention frequency), and additional statistical features capturing backstory complexity and semantic coherence.

B Hyperparameter Configuration

Random Forest hyperparameters were selected via grid search over n_estimators $\in \{100, 200, 300\}$ and max_depth $\in \{5, 10, 15\}$. XGBoost parameters (learning_rate, max_depth, subsample, colsample_bytree) were tuned through cross-validation. Logistic Regression regularization parameter C was optimized via stratified cross-validation to prevent overfitting while maintaining generalization performance.